IOAI International Olympiad in Artificial Intelligence

Konvolučné neurónové siete,

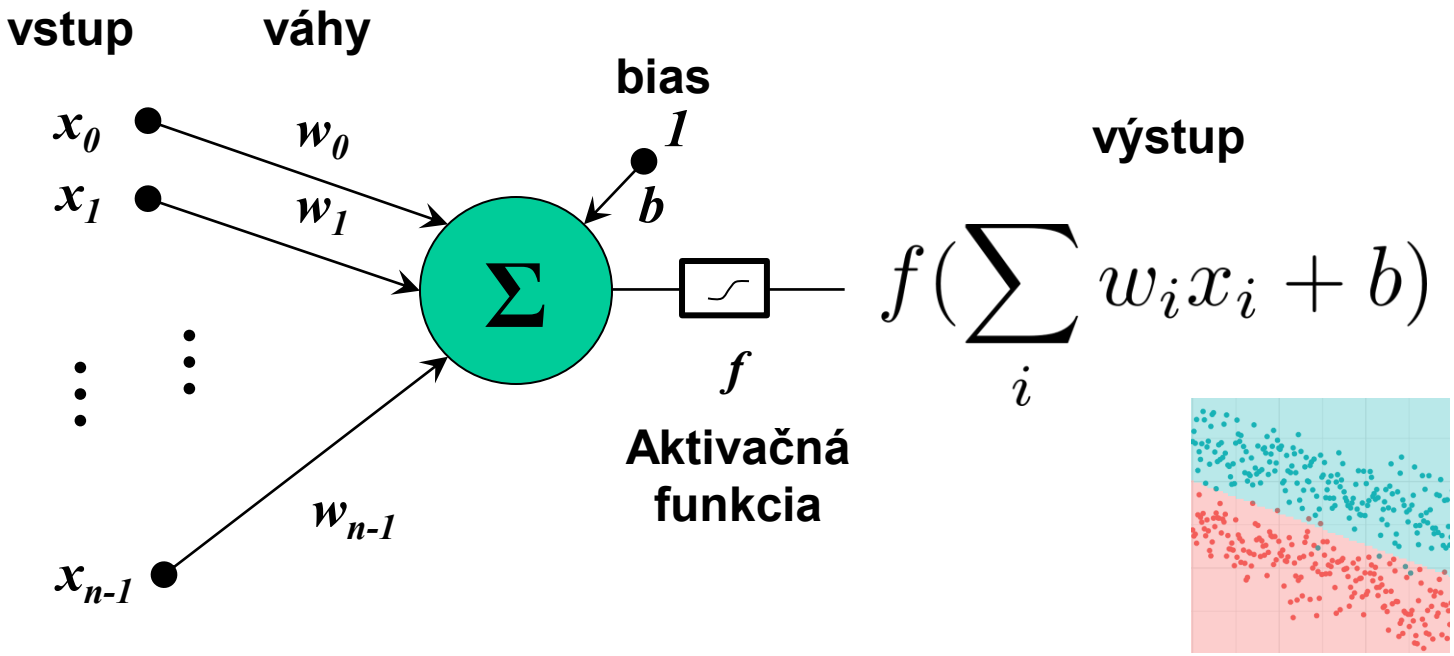
10.11.2026, 15:30-17:00

Andrej Lúčny

Ako sa od perceptronu, pre ktorý je spracovanie obrazu úloha s príliš veľkou dimenziou vstupných dát, dostaneme ku konvolučnej sieti, ktorá obraz spracúvať dokáže. Pochopíme, že dva bloky konvolučných vrstiev s kernelom 1x1 implementujú paralelne bežiacie perceptrony, zdieľajúce váhy, zatiaľ čo kernely 3x3 a viac zabezpečujú aj spoluprácu medzi nimi. Ako modelovú úlohu vyriešime odstránenie šumu.

`https://github.com/andylucny/OAI`

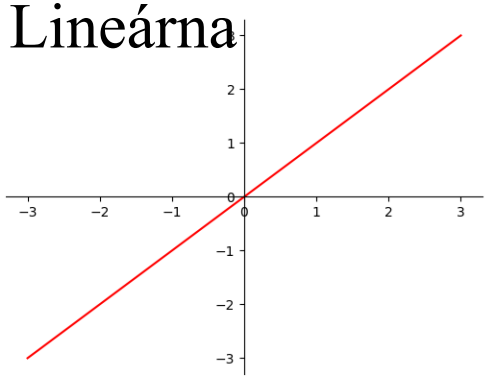
Neurón



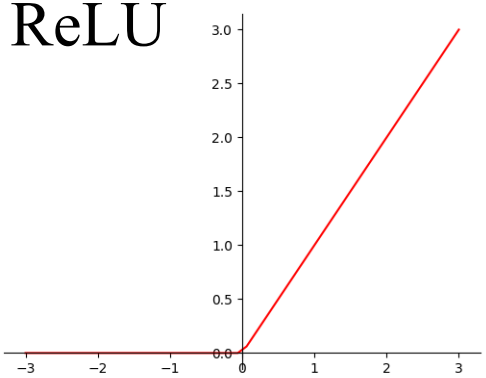
- Neurón počíta skalárny súčin vstupu s váhami, pripočíta bias a aplikuje aktivačnú funkciu
- Aktivačná funkcia môže byť:
  - Lineárna (lineárny regresor)
  - Sigmoida (logistický regresor, dáva hodnoty 0 až 1)
  - hyperbolický tangens (dáva hodnoty -1 až 1)

Aktivačné funkcie

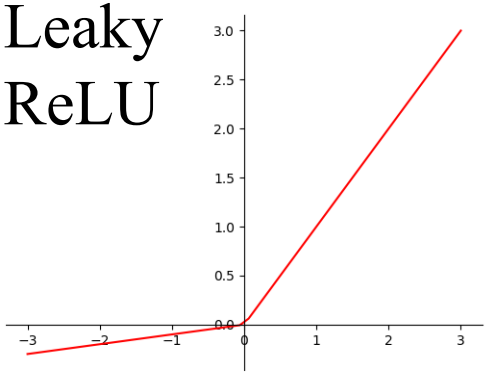
Lineárna



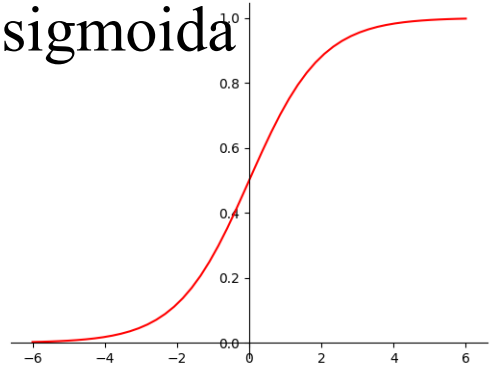
ReLU



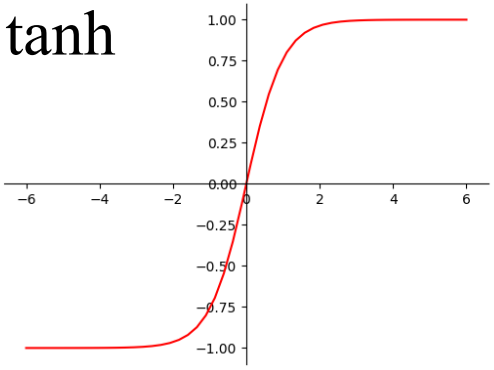
Leaky ReLU



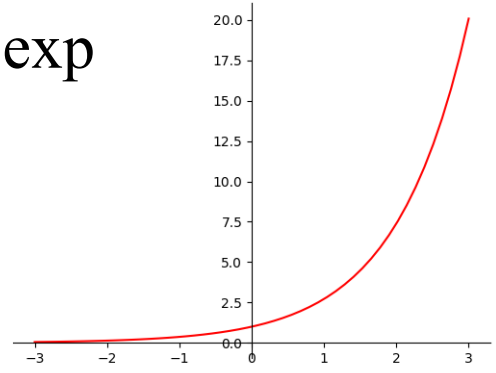
sigmoida



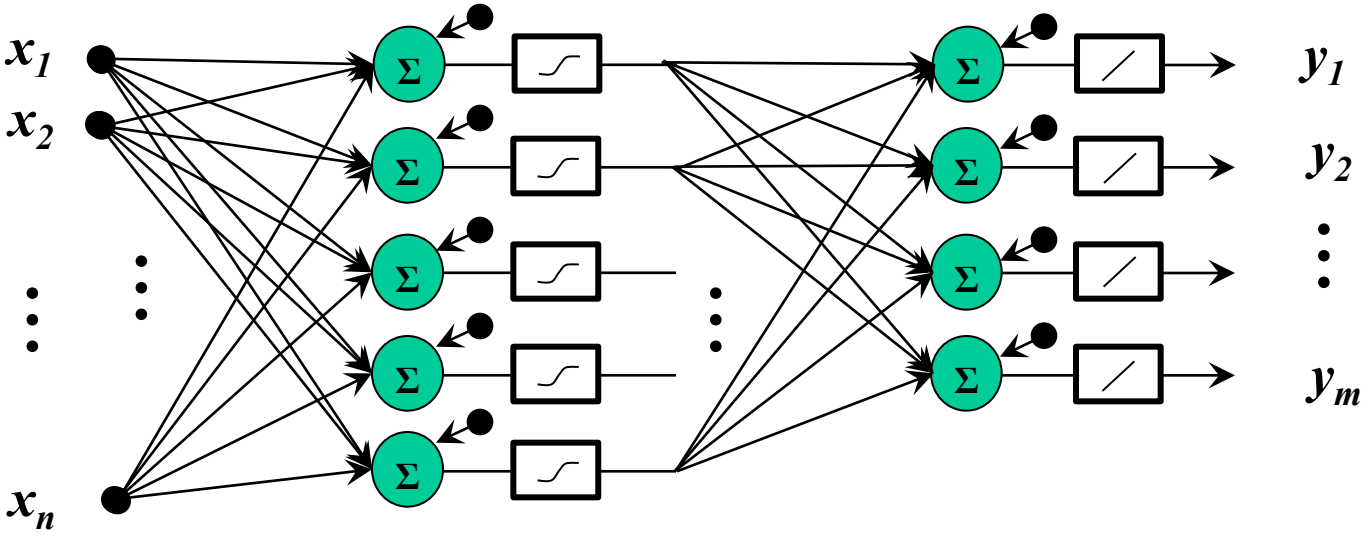
tanh



exp



Perceptron



- Trénovanie: algoritmus spätného šírenia (Back propagation)
  [1958 Rosenblat]
  [1986 Rumelhart, Hinton & Williams]
- matematicky bolo dokázané, že perceptron sa teoreticky dokáže s ľubovoľnou presnosťou naučiť akúkoľvek rovnomerne spojitú funkciu [1989 Cybenko] [1989 Hornik]

Čiernobiely obraz

rozlíšenie

$1 \times H \times W$

počet kanálov



rozsah:
0 .. 255
0.0 .. 1.0
-1.0 .. 1.0
-0.5 .. 0.5
-3 .. 3

OpenCV: $H \times W$

Pillow: $C \times H \times W$

Farebný obraz

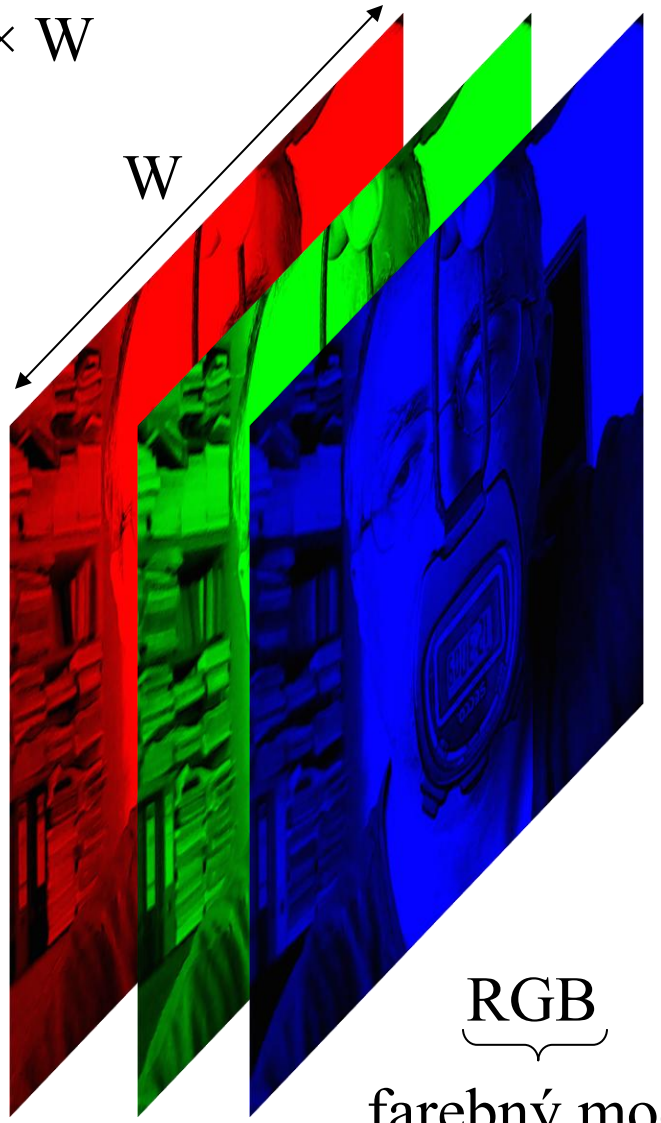
rozlíšenie

$C \times H \times W$

počet kanálov

W

H



RGB

farebný model

rozsah:

0 .. 255

0.0 .. 1.0

-1.0 .. 1.0

-0.5 .. 0.5

-3 .. 3

BGR

OpenCV: $H \times W \times C$

Pillow: $C \times H \times W$

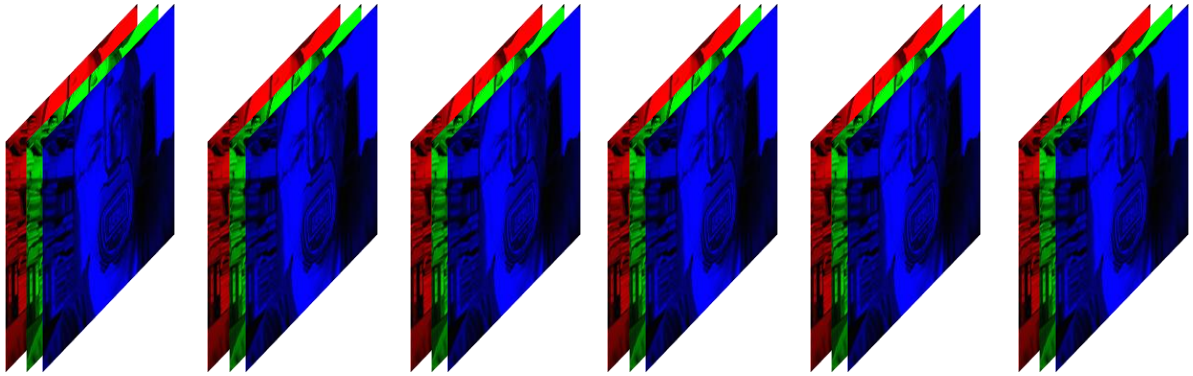
RGB



V súčasnosti sa používa taká normalizácia aby priemerná hodnota farebného kanála skrz všetky obrázky bola 0 a smerodajná odchýlka 1 (normálne rozdelenie)

Dávka

$B \times C \times H \times W$



vstupom neurónovej siete je vždy dávka (kvôli tréningu)
Keď potom používame sieť pre jeden vstup, dáváme na jej vstup tenzor $1 \times C \times H \times W$

```
import cv2
image = cv2.imread('image.jpg')
blob = cv2.blobFromImage(image,swapRB=True)
```

```
from PIL import Image
from torchvision import transforms
image = Image.open('image.jpg').convert("RGB")
blob = transform.ToTensor()(image).unsqueeze(0)
```

Hlavná myšlienka CNN

- Dimenzia obrazu ($>10^5$) je pre praktické použitie perceptronu příliš veľká
- Dimenziu obrazu potrebujeme nejako zmenšiť bez toho, že by sme stratili jeho informačnú hodnotu
- Typická CNN sa skladá z backbone a perceptronu
- Dnes si povieme: ako obraz spracúvať (pomocou kernelov)
- Nabudúce: ako redukovať jeho dimenziu bez straty informácie

Kernel

240x320x1



1x1x1

1.5

weight

0.15

bias

240x320x1



intenzita × váha + bias

Kernel

240x320x1



1x1x1

1.5

weight

0

bias

240x320x1



váha je kontrast

Kernel

240x320x1



1x1x1

1.0

weight

0.15

bias

bias je jas

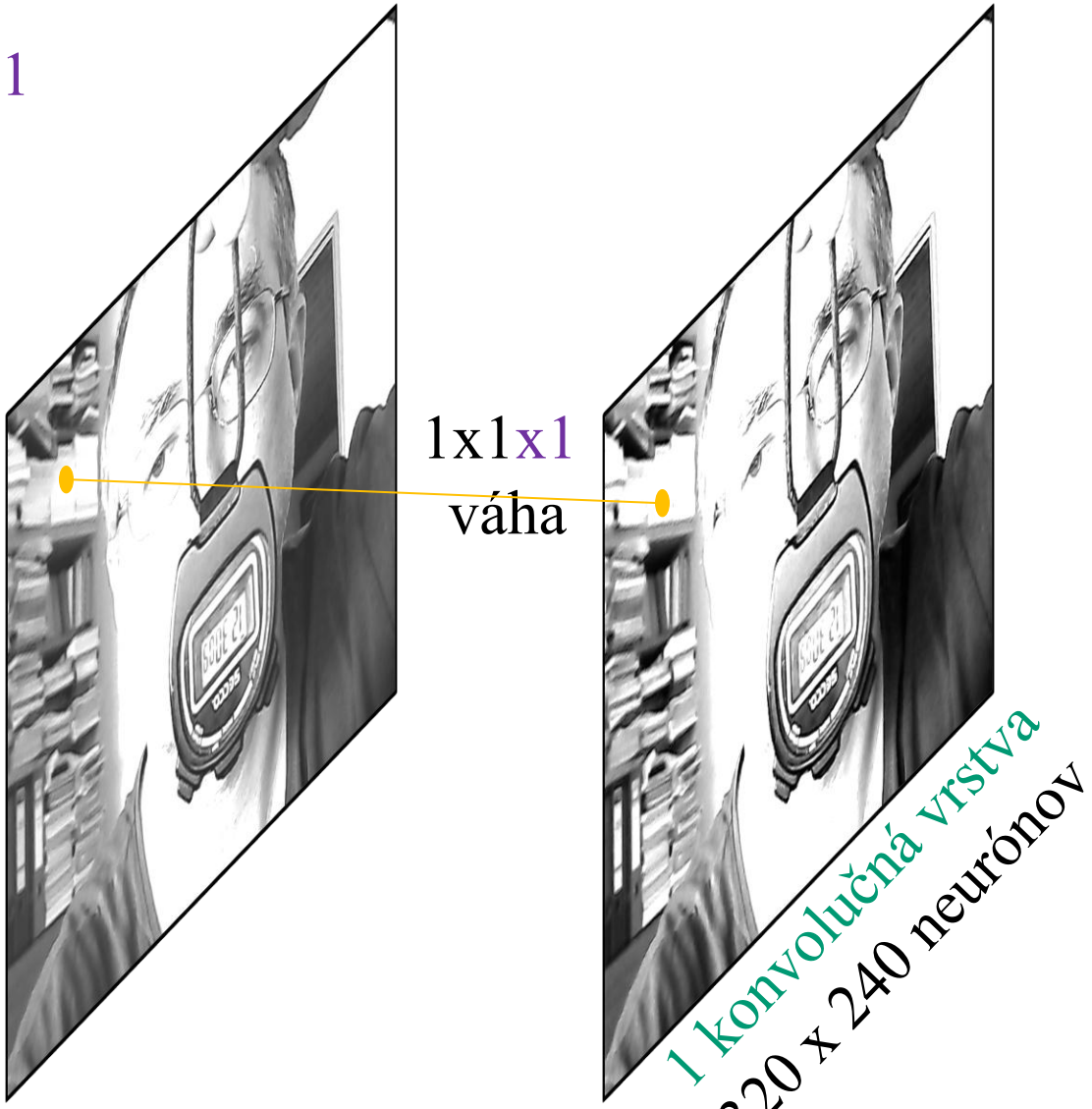
240x320x1



Konvolučná vrstva

320x240x1
vstup

dimenzia
vstupu
76800



všetky neuróny
zdieľajú
1 váhu a
1 bias

vrstva obsahuje
76800 neurónov
ale má len 2
parametre

320x240x1
výstup

aktivácia: lineárna

Tréning



vzorový vstup





vzorový výstup

1.5 kontrast

váha

0.15 jas

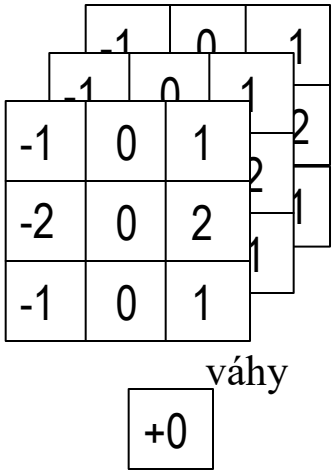
bias

Kernel

240x320x3



3x3x3

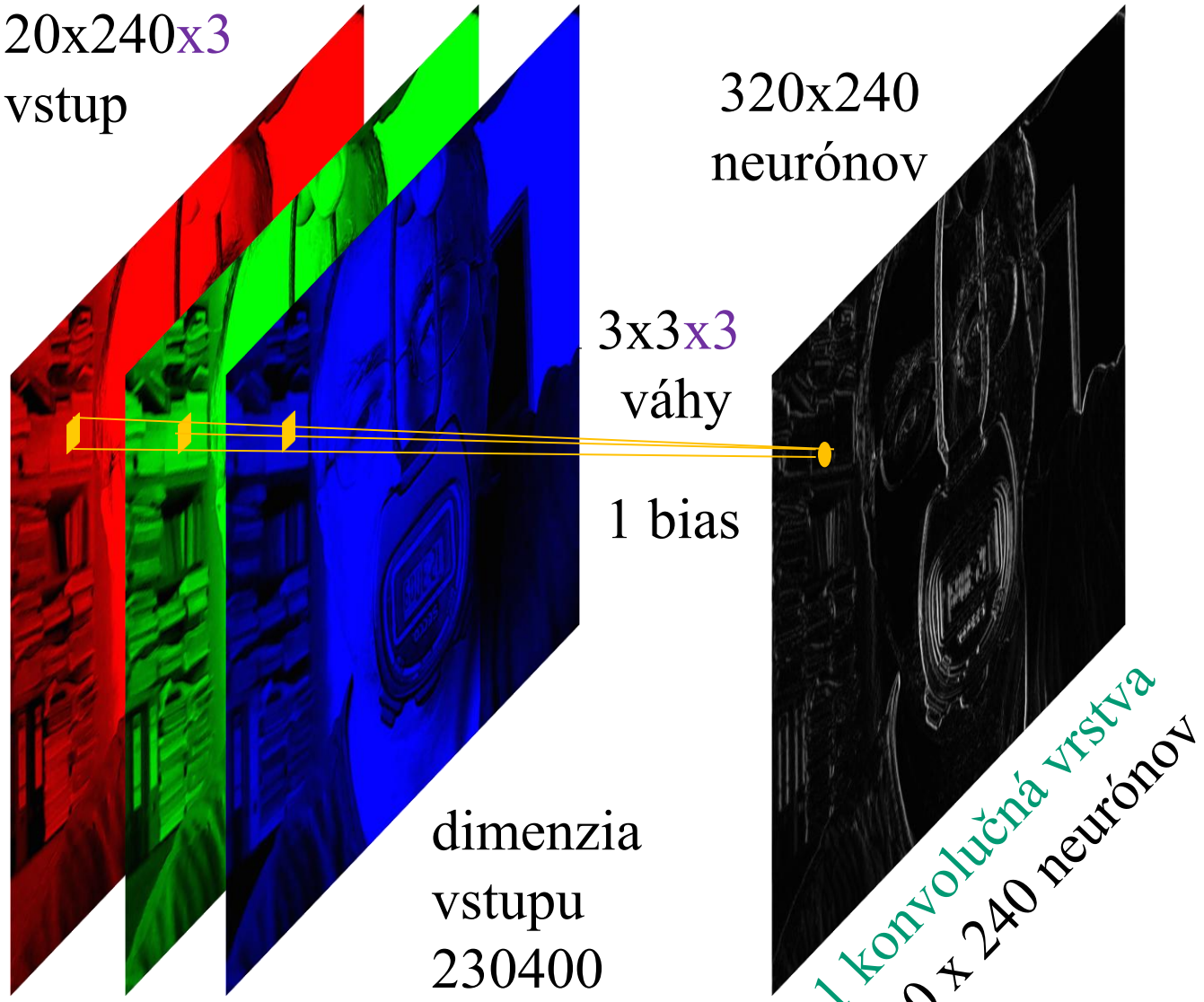


240x320x1



Sobelov (vertikálny) kernel dáva vertikálne hrany

Konvolučná vrstva



Všetky neuróny zdieľajú 27 váh a 1 bias

Vrstva obsahuje 76800 neufónov and 28 parameterov

320x240x1 výstup

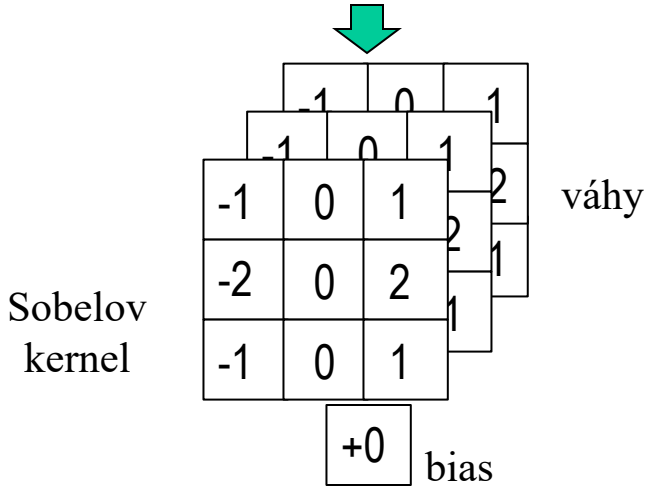
aktivácia: lineárna

Training

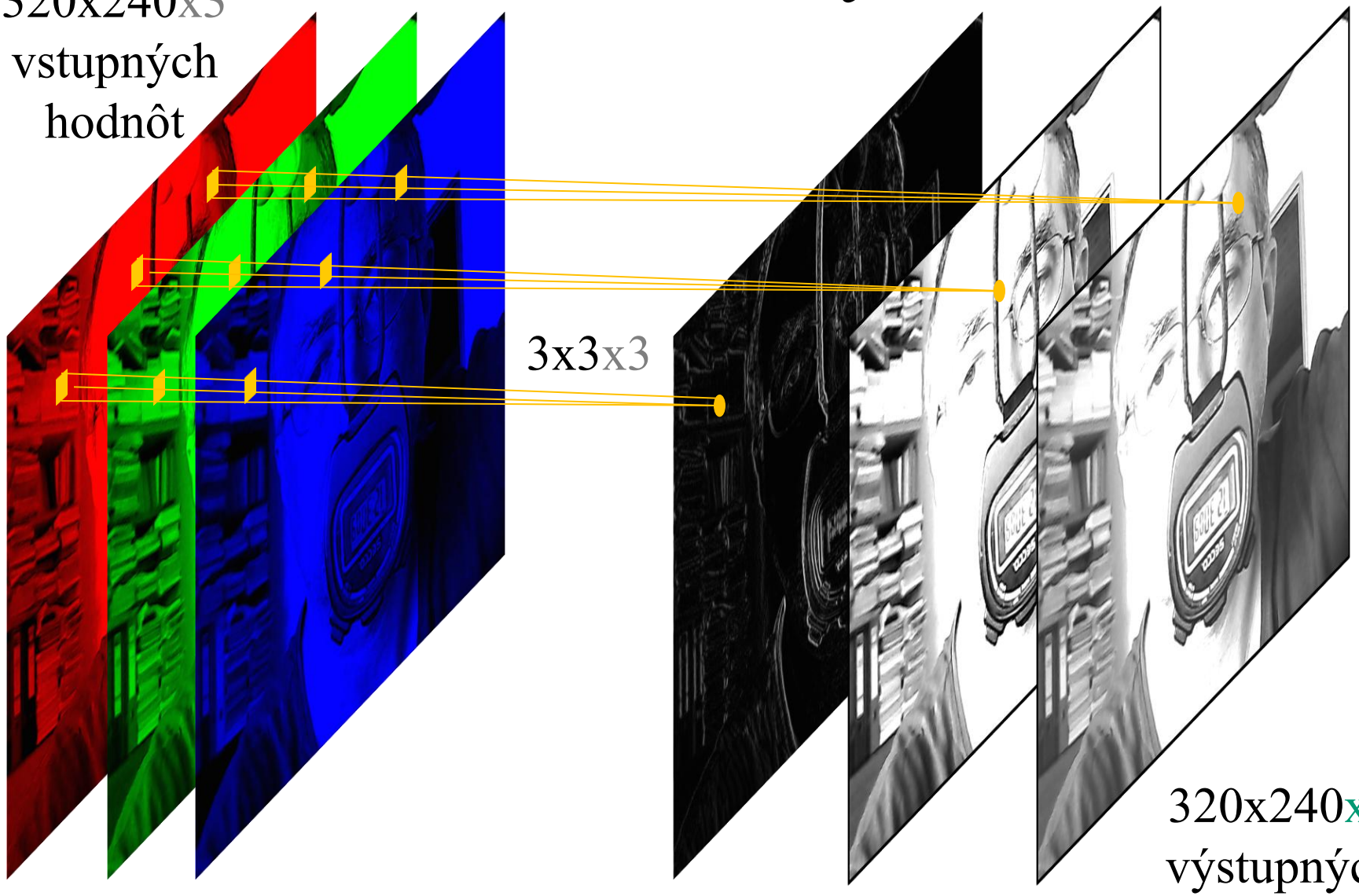






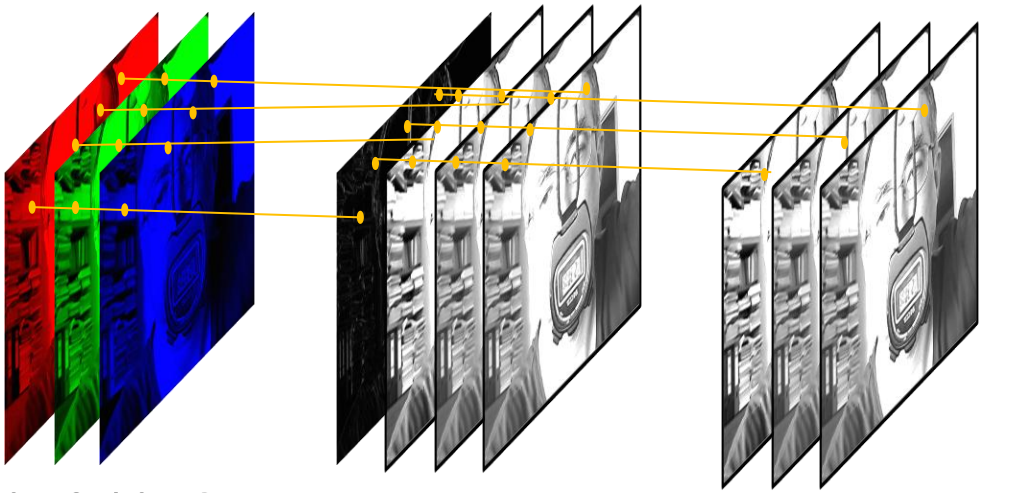


Blok konvolučných vrstiev



Typická aktivačná funkcia: ReLU alebo LeakeyReLU

Dva bloky konvolučných vrstiev

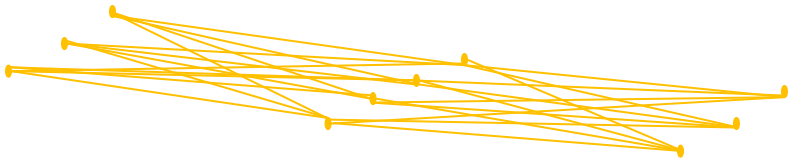


320x240x3
230400
pixelov

Blok 1
320x240x4
307200
neurónov
4x(3x1+1)
16
parametrov

Blok 2
320x240x3
230400
neurónov
3x(4x1+1)
15
parametrov

Dva bloky konvolučných vrstiev

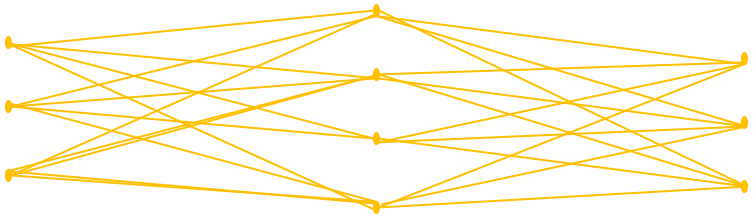


320x240x3
230400
pixelov

Blok 1
320x240x4
307200
neurónov
4x(3x1+1)
16
parametrov

Blok 2
320x240x3
230400
neurónov
3x(4x1+1)
15
parametrov

Dva bloky konvolučných vrstiev



3
vstupy
(kanály)

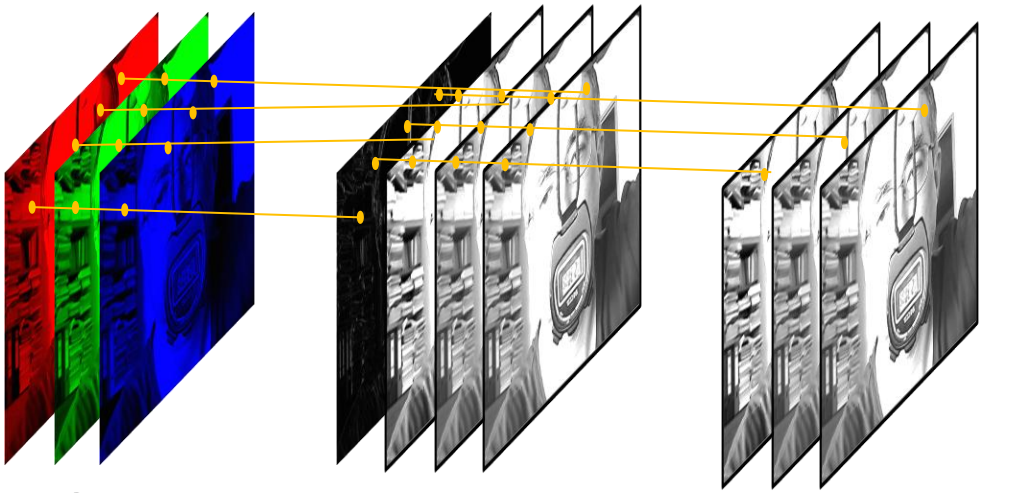
hidden
4
neuróny
4x(3+1)
16
parametrov

output
3
neuróny
3x(4+1)
15
parametrov

perceptron

Dva bloky konvolučných vrstiev

320x240x3
vstupných
hodnôt



3
vstupy
(kanály)

hidden
4
neuróny
4x(3+1)
16
parametrov

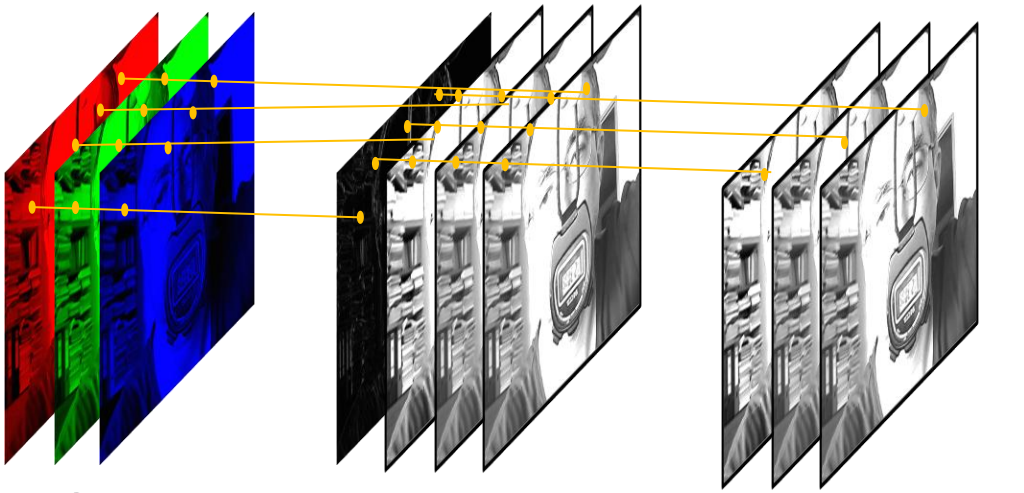
output
3
neuróny
3x(4+1)
15
parametrov

perceptron

koľko
takýchto
perceptronov
tam paralelne
beží?

Dva bloky konvolučných vrstiev

320x240x3
vstupných
hodnôt



3
vstupy
(kanály)

hidden
4
neuróny
4x(3+1)
16
parametrov

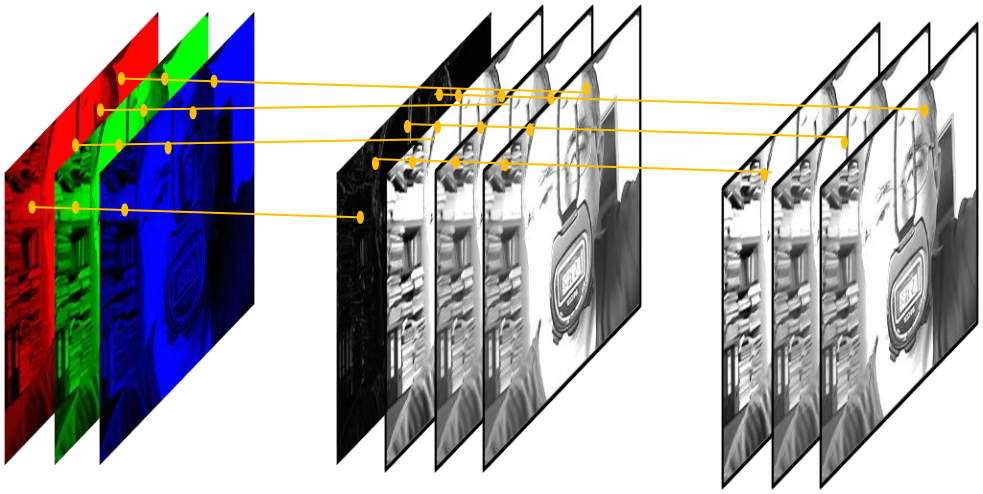
output
3
neuróny
3x(4+1)
15
parametrov

perceptron

koľko
takýchto
perceptronov
tam paralelne
beží?

320x240
76800

Dva bloky konvolučných vrstiev



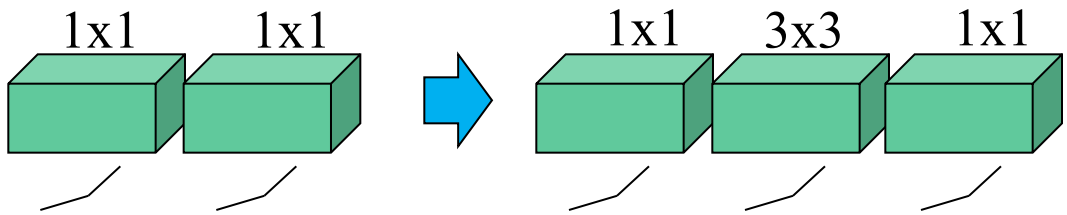
Dva bloky konvolučných vrstiev s kernelom 1x1 teda zodpovedajú paralelne bežiacim perceptronom zdieľajúcim váhy

A keďže používame nelineárne aktivácie (napríklad ReLU), ide o univerzálny aproximátor

Čo je vlastne CNN?

- Čo ak teraz použijeme kernel väčší ako 1x1, napríklad 3x3?

- Zabezpečí to výmenu informácii medzi susednými paralelne bežiacimi perceptrónmi.



Čo je vlastne CNN?

- CNN je teda sieť, kde paralelne púšťame rovnaké spolupracujúce perceptrony nad rôznymi miestami obrazu

- Tým premieňame obraz (mapa farieb) na mapu príznakov

Translačná invariantnosť

- Načo je dobré bežať takto paralelne kooperujúce perceptrony zdieľajúce váhy?

Úloha: odstránenie šumu

Vstup 3



Cieľ 3



Cieľ 3



PSNR	Kvalita rekonštrukcie
pod 20 dB	výrazné poškodenie
20–25 dB	viditeľné chyby
25–30 dB	prijateľný výsledok
30–35 dB	dobrá kvalita
35–40 dB	veľmi dobrá kvalita
nad 40 dB	rozdiel je často ťažko viditeľný

https://colab.research.google.com/github/andylucny/OAI/blob/master/01-convolution/denoising-outputs.ipynb